

yields "{ getattr open append lock ioctl }", therefore the above allow rule after expanding the macros becomes:

```
allow myapp_t myapp_log_t:file { open getattr read
ioctl lock append };
```

The permission macro "manage_file_perms" expands to "{ create open getattr setattr read write append rename link unlink ioctl lock }".

Thus, expanding all the permission macros, our example.te file shall appear as below:

```
#####
##
## Declarations
##
##
type myapp_t;
type myapp_exec_t;
domain_type(myapp_t)
domain_entry_file(myapp_t, myapp_exec_t)
#
type myapp_log_t;
logging_log_file(myapp_log_t)
#
type myapp_tmp_t;
files_tmp_file(myapp_tmp_t)
#
#####
##
## Myapp local policy
##
#expanded macros read_file_perms and append_file_perms
allow myapp_t myapp_log_t:file { open getattr read
ioctl lock append };
#expanded macro manage_file_perms
allow myapp_t myapp_tmp_t:file { create open
getattr setattr read write append rename link
unlink ioctl lock };
files_tmp_filetrans(myapp_t, myapp_tmp_t, file)
#
```

Capability macros

Let us now look at capability macros. Capability macros contain directives that expand into a module statement. Let us look at the first capability macro defined in the example.te file: domain_type(myapp_t). Like any procedure or function call in a programming language, arguments are passed in parenthesis.

The macro domain_type is defined in the file - /usr/share/selinux/devel/include/kernel/domain.if. It is defined as below:

```
#####
## <summary>
## Make the specified type usable as a
domain.
## </summary>
## <param name="type">
## <summary>
## Type to be used as a domain type.
## </summary>
## </param>
##
interface('domain_type',
# start with basic domain
domain_base_type($1)
')
#####
```

The comments above the macro declaration are very well documented. From the above, domain_type(myapp_t) expands to domain_base_type(myapp_t), which is again a macro. This is defined in the same file as:

```
interface('domain_base_type',
gen_require('
attribute domain;
')
typeattribute $1 domain;
')
```

The above macro introduces us to the attribute and typeattribute statements. We shall cover these in a subsequent article. Continuing with our macro expansion, the macro can be expanded to:

```
attribute domain;
typeattribute $1 domain;
```

Therefore, the macro domain_type(myapp_t) finally expands to:

```
attribute domain;
typeattribute myapp_t, domain;
```

Therefore, the above type enforcement file (example.te) can be rewritten as:

We now
have 4 of
the 7 RHCAs
in India
YOU could
be next!!!
Call us for
details of
next batch

other courses:

RHCE
RHCSS
JBoss
PHP
LAMP

**System Admin's
Tool Box**

Get trained at
FOSTERingLinux
The only
Open Source institute
by
System Administrators
&
Solution Architects

FOSTERingLinux
Free & Open Source Training Education & Research

SCO M-37, II Floor
Old DLF Colony, Sector - 14
Gurgaon - 122 001, Haryana
Tel: +91-124-4268187 / 4080880
Email: info@fosteringlinux.com
Website: www.fl.keenable.com

redhat
TRAINING PARTNER